Exercise 9 – Random Forests

Introduction to Machine Learning

Hint: Useful libraries

R

```
# Consider the following libraries for this exercise sheet:

library(mlr3verse)
library(rpart.plot)
library(data.table)
```

Python

```
# Consider the following libraries for this exercise sheet:

# general
import numpy as np
import pandas as pd

# plots
import matplotlib.pyplot as plt

# sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
```

Exercise 1: Random forest predictions

> Learning goals
>
> 1) Trace observations through the trees of a random forest
> 2) Compute random forest predictions by hand

The `wine` dataset contains results of a chemical analysis of wines grown in the same region in Italy, stemming from three different cultivars (`type` 1, 2 and 3). The aim is to classify the `type` of a wine, based on measurements such as its alcohol content, flavanoids and hue.

On these data, you have trained a random forest consisting of 3 trees of maximum depth 2. Consider the following subset of the training data:

observation	alcalinity	alcohol	flavanoids	hue	malic	phenols
1	11.4	14.75	3.69	1.25	1.73	3.10
2	25.0	13.40	0.96	0.67	4.60	1.98
3	17.4	13.94	3.54	1.12	1.73	2.88

The model information below was created with `ranger::treeInfo()`, which assigns observations with values larger than `splitval` to the right child node in each split. For terminal nodes, `prediction` states the predicted class.

Tree 1:

nodeID	leftChild	rightChild	splitvarName	splitval	terminal	prediction
0	1	2	phenols	1.94	FALSE	NA
1	3	4	alcohol	12.43	FALSE	NA
2	5	6	alcohol	13.04	FALSE	NA
3	NA	NA	NA	NA	TRUE	2
4	NA	NA	NA	NA	TRUE	3
5	NA	NA	NA	NA	TRUE	2
6	NA	NA	NA	NA	TRUE	1

Tree 2:

nodeID	leftChild	rightChild	splitvarName	splitval	terminal	prediction
0	1	2	alcohol	12.78	FALSE	NA
1	3	4	hue	0.68	FALSE	NA
2	5	6	flavanoids	2.18	FALSE	NA
3	NA	NA	NA	NA	TRUE	3
4	NA	NA	NA	NA	TRUE	2
5	NA	NA	NA	NA	TRUE	3
6	NA	NA	NA	NA	TRUE	1

Tree 3:

nodeID	leftChild	rightChild	splitvarName	splitval	terminal	prediction
0	1	2	alcohol	12.79	FALSE	NA
1	3	4	phenols	2.01	FALSE	NA
2	5	6	phenols	2.28	FALSE	NA
3	NA	NA	NA	NA	TRUE	2
4	NA	NA	NA	NA	TRUE	2
5	NA	NA	NA	NA	TRUE	3
6	NA	NA	NA	NA	TRUE	1

For the subset of the training data and the random forest model given above,

a) find the terminal node of each tree the observations are placed in,

b) state each tree's prediction for the three observations and derive the prediction of the random forest.

Exercise 2: Classifying spam

Learning goals

1) Understand the instability of single decision trees
2) Apply RF to data for prediction, OOB error estimation & feature importance computation

In this exercise, we work with the `spam` dataset, a binary classification task where the aim is to classify an e-mail as spam or non-spam, based on 57 features describing the frequency of certain words and characters. You can set up the data as follows:

R

```
library(mlr3verse)
task_spam <- tsk("spam")
```

Python

Read spam.csv:

```
data_spam = pd.read_csv("spam.csv")
data_spam.drop(data_spam.columns[[0]], axis=1, inplace=True)
X_spam = data_spam.copy()
y_spam = X_spam.pop("type")
```

a) Use a decision tree to predict `spam`. Re-fit the tree using two random subsets of the data (each comprising 60% of observations). How stable are the trees?

Hint

R

Use `rpart.plot()` from the package `rpart.plot` to visualize the trees.

Python

Use `from sklearn.tree import plot_tree` to visualize the trees.

b) Forests come with a built-in estimate of their generalization ability via the out-of-bag (OOB) error: each tree is evaluated on the observations that did not end up in its bootstrap sample. Use the random forest learner (R: `classif.ranger`, Python: `RandomForestClassifier()`) to fit the model and state the OOB error.

c) You are interested in which variables have the greatest influence on the prediction quality. Explain how to determine this in a permutation-based approach and compute the importance scores for the `spam` data.

Hint

R

Use an adequate variable importance filter as described here.

Python

Choose an adequate importance measure as described here.

Exercise 3: Out-of-bag error by hand

The whole exercise is only for lecture group B!

Learning goals

1. Identify out-of-bag (OOB) observations from bootstrap samples
2. Compute the OOB error of a bagging ensemble by hand
3. Explain why the OOB error estimates generalization error

A colleague has trained a small bagging ensemble to detect spam e-mails: 3 classification trees, each fitted on a bootstrap sample drawn with replacement from the $n = 6$ training e-mails, labeled with $y = 1$ for spam and $y = 0$ for non-spam.

The first table shows the composition of the bootstrap samples, i.e., how often each e-mail was drawn into the bootstrap sample of each tree:

e-mail i	$y^{(i)}$	Tree 1	Tree 2	Tree 3
1	1	2	0	1
2	0	1	1	0
3	1	1	2	0
4	0	0	1	2
5	1	2	2	2
6	0	0	0	1

The second table shows the prediction $\hat{y}$ of each (already fitted) tree for all six e-mails:

e-mail i	Tree 1	Tree 2	Tree 3
1	1	1	1
2	0	0	1
3	1	1	1
4	0	0	0
5	1	1	1
6	1	1	0

a) For each tree, determine which e-mails are out of bag. Also explain why an e-mail can appear twice in a bootstrap sample.

b) Compute the OOB prediction for each e-mail where possible: aggregate the predictions of all trees for which the e-mail is out of bag. Which e-mail does not receive an OOB prediction, and why?

c) Using your results from b), compute the OOB error of the ensemble.

d) The trees in this ensemble classify the e-mails in their own bootstrap sample perfectly: verify this for one tree of your choice. Why is the OOB error from c) nevertheless the more realistic estimate of how well the ensemble will perform on new e-mails?

Exercise 4: Bagging and proximities

The whole exercise is only for lecture group A!

Learning goals

1. Understand benefit of bagging from a mathematical perspective
2. Solve "show that…"-type exercises
3. Handle expectations over random variables
4. Compute proximities

In this exercise, we briefly revisit why bagging is a useful technique to stabilize predictions and take a closer look at two random forest particulars: out-of-bag observations and proximities.

a) Random forests provide a built-in estimate of their generalization ability via the out-of-bag (OOB) error (cf. Exercise 2): each tree can be evaluated on the observations that did not end up in its bootstrap sample. Show that the probability for an observation to be OOB in an arbitrary bootstrap sample converges to $\frac{1}{e}$.

b) For a fixed observation $(\mathbf{x}, y)$, show that the expected quadratic loss over individual base learner predictions $b^{[m]}(\mathbf{x})$ is larger than or equal to the quadratic loss of the prediction $f^{[M]}(\mathbf{x})$ of a size-M ensemble. You can consider all hyperparameters of the base learners and the ensemble fixed.

Hint

Use the law of total expectation ("Verschiebungssatz der Varianz": $\mathsf{Var}(Z) = \mathbb{E}(Z^2) - (\mathbb{E}(Z))^2 \iff \mathbb{E}(Z^2) = \mathsf{Var}(Z) + (\mathbb{E}(Z))^2$, where $\mathsf{Var}(Z) \geq 0$ by definition.) stating $\mathbb{E}(Z^2) \geq (\mathbb{E}(Z))^2$ for a random variable Z.

c) After training the random forest from Exercise 1, you wish to determine how similar the three observations are in the eyes of the forest. Compute their pairwise proximities, using your results from Exercise 1 a).